

Informationstechnologie I - Skript zu Termin 1:

Einführung in die C#-Programmierung

1. Einführung: Von der Aufgabe zum Programm

Ein Programm beschreibt eine Folge von Anweisungen, die ein Computer ausführt. Damit lassen sich beispielsweise Berechnungen durchführen, Eingaben verarbeiten oder wiederkehrende Aufgaben automatisieren. Der Computer benötigt dafür genaue Angaben: Welche Werte stehen zur Verfügung? Was soll mit ihnen geschehen? Welches Ergebnis soll ausgegeben werden?

Eine genaue Schrittfolge zur Lösung einer Aufgabe wird als **Algorithmus** bezeichnet. Ein solcher Algorithmus lässt sich zunächst in Alltagssprache formulieren. Für die Aufgabe „Alter in Tagen berechnen“ könnte er so aussehen:

1. Das Alter in Jahren entgegennehmen.
2. Das Alter mit 365 multiplizieren.
3. Das Ergebnis in Tagen anzeigen.

Für ein Alter von 20 Jahren ergibt sich $20 * 365 = 7300$. Die Berechnung ist eine Näherung, da Schaltjahre und das genaue Geburtsdatum unberücksichtigt bleiben.

Dieses Beispiel folgt einem grundlegenden Muster der Datenverarbeitung: **Eingabe** → **Verarbeitung** → **Ausgabe**. Die Eingabe liefert die benötigten Daten, die Verarbeitung berechnet ein Ergebnis und die Ausgabe macht es sichtbar.

In dieser Vorlesung wird die Programmiersprache **C#** verwendet, um solche Abläufe als ausführbaren Code zu beschreiben. Der Schwerpunkt liegt zunächst auf kleinen Konsolenprogrammen. Die **Konsole** ist der Bereich, in dem ein Programm Texte ausgibt und Eingaben entgegennimmt. Programmiererfahrung wird für die folgenden Beispiele nicht vorausgesetzt.

2. Das erste C#-Programm

Ein einfaches Programm besteht bereits aus zwei Anweisungen:

```
Console.WriteLine("Hallo!");  
Console.WriteLine("Programmierung mit C#.");
```

Die Ausgabe lautet:

```
Hallo!  
Programmierung mit C#.
```

Die Anweisungen werden hier **von oben nach unten** ausgeführt. Zunächst erscheint **Hallo!**, danach **Programmierung mit C#.** auf einer neuen Zeile. Der **Quelltext** ist der Code, der im Editor geschrieben

wird. Die **Ausgabe** entsteht erst bei der Ausführung des Programms.

Aufbau einer Anweisung

Die Anweisung `Console.WriteLine("Hallo!");` besteht aus mehreren Teilen:

Bestandteil	Bedeutung
<code>Console.WriteLine</code>	Ruft eine Methode auf, die Text oder einen Wert mit anschließendem Zeilenumbruch ausgibt.
<code>(...)</code>	Enthält den Text oder Wert, der ausgegeben werden soll.
<code>"Hallo!"</code>	Ein Textwert; die Anführungszeichen begrenzen den Text im Quellcode.
<code>;</code>	Beendet die Anweisung.

Eine **Methode** ist eine benannte Funktion, die eine bestimmte Aufgabe erledigt. Hier wird eine bereits vorhandene Methode benutzt. Wie eigene Methoden geschrieben werden, wird in einem späteren Termin behandelt.

Die Anführungszeichen, die runden Klammern und das Semikolon gehören zur Schreibweise des Codes und erscheinen nicht in der Ausgabe. Sie erfüllen unterschiedliche Aufgaben und müssen an den richtigen Stellen stehen. Das Semikolon beendet beispielsweise eine Ausgabe oder eine Zuweisung. Später werden auch Codeblöcke betrachtet, deren geschweifte Klammern nicht mit einem Semikolon abgeschlossen werden.

Arbeiten im vorbereiteten Projekt

Die Vorlesung verwendet die vorbereitete GitHub-Vorlage **DHBW-Vorlesung-Codespace**. Der Codespace stellt den Editor und das .NET SDK bereit, mit dem C#-Code übersetzt und ausgeführt werden kann. Er kann unter folgender URL gefunden werden: <https://github.com/Rearth/DHBW-Vorlesung-Codespace>. Zum öffnen wird ein kostenloses github konto benötigt. Falls dies nicht funktioniert oder ohne konto gearbeitet werden soll, kann auch folgende Seite genutzt werden: <https://www.jdoodle.com/compile-c-sharp-online>. Alternativ ist auch das installieren des .NET SDKs sowie einer Entwicklungsumgebung wie Visual Studio möglich.

Öffnen Sie die vorbereitete C#-Datei, bearbeiten Sie den Beispielcode und speichern Sie Ihre Änderungen. Starten Sie das Programm anschließend über den **Run-Button** ► in VS Code und prüfen Sie die Ausgabe. Im aktuellen Projekt heißt die vorbereitete Datei `Termin_1.cs`.

Die Beispiele in diesem Skript verwenden **Top-Level-Statements**: Die Anweisungen stehen direkt in der Datei. Für diese Beispiele müssen keine eigene Klasse und keine `Main`-Methode geschrieben werden.

Verwenden Sie die Beispiele jeweils einzeln als Inhalt der vorbereiteten Datei. Werden mehrere Beispiele einfach untereinander eingefügt, können unter anderem doppelte Variablennamen entstehen. Ein sinnvoller Arbeitsablauf lautet: **Ausgabe vorhersagen → Code ausführen → Ergebnis erklären**.

3. Variablen: Typ, Name und Wert

Betrachten wir zunächst eine Berechnung ohne Variablen:

```
Console.WriteLine(20);  
Console.WriteLine(20 * 365);
```

Die Zahl 20 bezeichnet hier ein Alter. Diese Bedeutung ist im Code jedoch nicht unmittelbar erkennbar. Außerdem müsste die Zahl bei einer Änderung an beiden Stellen angepasst werden.

Eine **Variable** speichert einen Wert unter einem Namen. Über diesen Namen kann der aktuelle Wert an mehreren Stellen verwendet werden:

```
int alter = 20;  
Console.WriteLine(alter);  
Console.WriteLine(alter * 365);
```

Nun wird das Alter einmal festgelegt und anschließend sowohl für die Ausgabe als auch für die Berechnung gelesen.

Eine Variable lässt sich vereinfacht mit einem **beschrifteten Speicherfach** vergleichen. Der Name steht auf dem Fach, der aktuelle Wert liegt darin und der Datentyp legt fest, welche Werte hineinpassen.

Deklaration und Initialisierung

Die Anweisung `int alter = 20;` verbindet zwei Schritte:

- **Deklarieren:** Eine Variable mit einem Namen und einem Datentyp anlegen.
- **Initialisieren:** Der Variable ihren ersten Wert zuweisen.

Die allgemeine Form lautet:

```
Datentyp Variablenname = Startwert;
```

Im Beispiel ist `int` der Datentyp für ganze Zahlen, `alter` der Name und `20` der Startwert. Das Zeichen `=` weist den Wert zu. In den folgenden Beispielen erhalten Variablen direkt beim Anlegen einen Startwert.

Variablen und Literale unterscheiden

Ein direkt im Quellcode geschriebener Wert wird als **Literal** bezeichnet. Beispiele sind `20`, `"Hallo!"` oder `true`.

```
int alter = 20;  
  
Console.WriteLine(alter); // Liest den aktuellen Wert der Variable.  
Console.WriteLine("alter"); // Gibt den wörtlichen Text aus.  
Console.WriteLine(20); // Gibt das Zahlenliteral aus.
```

Die Ausgabe lautet:

```
20
alter
20
```

Ohne Anführungszeichen wird bei `alter` der Variablenwert gelesen. **Mit Anführungszeichen** ist `"alter"` ein Textwert. Diese Unterscheidung ist besonders wichtig, wenn das Programm statt des erwarteten Wertes nur den Namen einer Variable ausgibt.

4. Zuweisungen und Veränderungen von Werten

Nach der Deklaration kann eine Variable einen neuen Wert erhalten. Dabei wird der Datentyp nicht erneut geschrieben:

```
int punkte = 10; // Variable anlegen und initialisieren.
punkte = 14;    // Den bisherigen Wert durch 14 ersetzen.
```

Eine Zuweisung folgt immer demselben Prinzip: **Zuerst wird der Ausdruck rechts von = ausgewertet, danach wird das Ergebnis in der Variable links gespeichert.** Ein Ausdruck kann beispielsweise ein einzelner Wert, eine Variable oder eine Berechnung sein.

```
int punkte = 14;
punkte = punkte + 3;
Console.WriteLine(punkte); // Ausgabe: 17
```

Rechts wird zunächst der bisherige Wert `14` gelesen. Die Rechnung `14 + 3` ergibt `17`. Erst dann ersetzt die Zuweisung den bisherigen Wert von `punkte` durch `17`.

Das Zeichen `=` beschreibt hier eine Aktion. Es ist keine mathematische Gleichung: Die Anweisung `punkte = punkte + 3;` bedeutet „Erhöhe den gespeicherten Wert um drei“.

Werte kopieren: Schritt für Schritt

Das folgende Programm zeigt mehrere Zuweisungen innerhalb eines Programmlaufs:

```
int punkte = 10;
int kopie = punkte;
punkte = 14;
punkte = punkte + 3;
kopie = punkte + kopie;
Console.WriteLine(kopie);
```

Die Variablenwerte entwickeln sich wie folgt:

Schritt	Ausgeführte Anweisung	punkte danach	kopie danach
1	<code>int punkte = 10;</code>	10	Noch nicht angelegt
2	<code>int kopie = punkte;</code>	10	10
3	<code>punkte = 14;</code>	14	10
4	<code>punkte = punkte + 3;</code>	17	10
5	<code>kopie = punkte + kopie;</code>	17	27
6	<code>Console.WriteLine(kopie);</code>	17	27

Beim Anlegen von **kopie** wird der **aktuelle int-Wert** von **punkte** übernommen. Es entsteht keine dauerhafte Verbindung zwischen diesen beiden Variablen. Deshalb bleibt **kopie** in Schritt 3 bei **10**, obwohl **punkte** auf **14** gesetzt wird.

In Schritt 5 werden rechts die bisherigen Werte **17** und **10** gelesen und addiert. Das Ergebnis **27** wird anschließend in **kopie** gespeichert. Die Ausgabe in Schritt 6 liest diesen Wert, verändert aber keine der Variablen.

Berechnete Ergebnisse aktualisieren sich nicht automatisch

Auch eine gespeicherte Berechnung ist ein Wert, keine dauerhaft hinterlegte Formel:

```
int anzahl = 3;
double einzelpreis = 2.50;
double gesamtpreis = anzahl * einzelpreis;

Console.WriteLine(gesamtpreis); // 7.5

anzahl = anzahl + 2;
Console.WriteLine(gesamtpreis); // Weiterhin 7.5

gesamtpreis = anzahl * einzelpreis;
Console.WriteLine(gesamtpreis); // Jetzt 12.5
```

Nach der ersten Berechnung enthält **gesamtpreis** den Wert **7.5**. Die Veränderung von **anzahl** auf **5** betrifft zunächst nur diese Variable. Erst die erneute Berechnung und Zuweisung verändert **gesamtpreis** auf **12.5**.

Dieses Verhalten unterscheidet sich beispielsweise von einer Formel in einer Tabellenkalkulation, die bei geänderten Eingabewerten automatisch neu berechnet wird. Im Programm muss die erneute Berechnung ausdrücklich ausgeführt werden. Die Zahlenausgabe kann abhängig von der Laufzeitumgebung ein Komma statt eines Punktes verwenden.

5. Grundlegende Datentypen

Der **Datentyp** bestimmt, welche Art von Werten eine Variable speichern kann und welche Operationen damit möglich sind. Eine Teilnehmerzahl, ein Name und eine Ja/Nein-Angabe benötigen unterschiedliche

Typen.

Datentypen im Überblick

Ein **Bit** kann einen von zwei Zuständen darstellen. Acht Bit bilden ein **Byte**. Mit mehr Bits lassen sich mehr unterschiedliche Werte darstellen. Daher besitzen Ganzzahltypen mit größerer Bitbreite größere Wertebereiche.

Typ	Größe	Wertebereich / Inhalt
<code>byte</code>	8 Bit	Ganze Zahlen von <code>0</code> bis <code>255</code> , ohne Vorzeichen
<code>short</code>	16 Bit	Ganze Zahlen von -2^{15} bis $2^{15} - 1$
<code>int</code>	32 Bit	Ganze Zahlen von -2^{31} bis $2^{31} - 1$
<code>long</code>	64 Bit	Ganze Zahlen von -2^{63} bis $2^{63} - 1$
<code>char</code>	16 Bit	Eine UTF-16-Codeeinheit, z. B. <code>'A'</code>
<code>float</code>	32 Bit	Gleitkommazahl; bis ca. $\pm 3,4 \times 10^{38}$; ca. 6–9 gültige Ziffern
<code>double</code>	64 Bit	Gleitkommazahl; bis ca. $\pm 1,8 \times 10^{308}$; ca. 15–17 gültige Ziffern
<code>decimal</code>	128 Bit	Dezimalzahl; bis ca. $\pm 7,9 \times 10^{28}$; ca. 28–29 gültige Ziffern
<code>bool</code>	8 Bit	Die Wahrheitswerte <code>true</code> oder <code>false</code>
<code>string</code>	Variabel	Text, z. B. <code>"Inhalt"</code>

Die Größen beschreiben die jeweiligen Werte und nicht den gesamten Verwaltungsaufwand einer Variable. Ein `bool` besitzt trotz seiner Größe nur zwei logische Werte. Ein `string` hat je nach Inhalt eine unterschiedliche Größe.

Die Tabelle dient als Nachschlagehilfe. Für viele einfache Ganzzahlberechnungen genügt `int`, dessen Bereich von $-2\,147\,483\,648$ bis $2\,147\,483\,647$ reicht. `short`, `int` und `long` können auch negative Werte speichern. Weitere Ganzzahltypen sind `sbyte` sowie die vorzeichenlosen Varianten `ushort`, `uint` und `ulong`.

Zahlen, Texte und Wahrheitswerte

```
int teilnehmerzahl = 24;
string raum = "A204";
char gruppenkennzeichen = 'B';
float temperatur = 21.5F;
bool veranstaltungAktiv = true;
```

- **int**: Speichert ganze Zahlen, beispielsweise `24`, `0` oder `-3`.
- **string**: Speichert Text. Textliterale stehen in doppelten Anführungszeichen.
- **char**: Speichert eine UTF-16-Codeeinheit. Einfache Zeichen wie `'B'` stehen in einfachen Anführungszeichen. Manche sichtbaren Unicode-Zeichen benötigen mehrere Codeeinheiten; für Texte wird `string` verwendet.

- **float und double**: Speichern binäre Gleitkommazahlen. Sie erlauben große Wertebereiche, besitzen aber eine begrenzte Genauigkeit.
- **decimal**: Eignet sich für Dezimalrechnungen, beispielsweise mit Geldbeträgen. Auch dieser Typ besitzt Grenzen und kann Rundungen erfordern.
- **bool**: Speichert **true** (wahr) oder **false** (falsch), etwa für eine Entscheidung oder einen Zustand.

Die Genauigkeit von Gleitkommazahlen wird in **gültigen beziehungsweise signifikanten Ziffern** angegeben. Damit sind Ziffern der gesamten Zahl gemeint, nicht eine feste Anzahl von Nachkommastellen. Die Einführung verwendet überwiegend **int** und **double**; die genaue Darstellung von Zahlen wird später vertieft.

Literale und Suffixe

Auch ein Literal besitzt einen Datentyp. Kleine ganze Zahlen wie **123** haben ohne Suffix den Typ **int**; ein Literal mit Dezimalpunkt wie **123.45** hat standardmäßig den Typ **double**. **true** und **false** sind **bool**-Literale und stehen ohne Anführungszeichen im Code.

Im C#-Quellcode ist das Dezimaltrennzeichen immer ein Punkt. Beispielsweise wird **2.50** geschrieben, auch wenn die Umgebung die Zahl später als **2,5** ausgibt.

Ein **Suffix**, also ein angehängter Buchstabe, legt den Typ eines Zahlenliterals gezielt fest:

```
long anzahl = 123L;  
float temperatur = 36.6F;  
double laenge = 123.45D;  
decimal betrag = 123.45M;
```

Das **D** bei **double** ist optional. Bei **float** ist das **F** für ein Literal wie **36.6F** erforderlich, weil **36.6** allein den Typ **double** hat und nicht automatisch in **float** umgewandelt wird. Große Suffixbuchstaben sind gut lesbar; insbesondere lässt sich **L** leichter von der Ziffer **1** unterscheiden als ein kleines **l**.

Typbindung und Compilerprüfung

Der Typ einer Variable bleibt nach ihrer Deklaration fest:

```
int anzahl = 3;  
anzahl = 4; // Gültig: Der neue Wert passt zum Typ int.
```

Die folgenden Zuweisungen sind **absichtlich ungültig**; sie zeigen typische Typfehler und dürfen so nicht als ausführbares Beispiel übernommen werden:

```
anzahl = "vier"; // Fehler: Text passt nicht zu int.  
anzahl = 4.5;    // Fehler: double wird nicht automatisch in int  
umgewandelt.
```

Der **Compiler** übersetzt den Quellcode und prüft dabei unter anderem die Schreibweise und die Verwendung der Datentypen. Solche Fehler werden bereits vor der Ausführung gemeldet. Manche Umwandlungen sind dagegen automatisch erlaubt, etwa `double wert = 123;`: Hier wird ein `int`-Wert in einen `double`-Wert umgewandelt.

6. Variablennamen und Kommentare

Ein guter Variablenname erklärt die Bedeutung eines Wertes. `alterInJahren` ist verständlicher als `x`, und `alterInTagen` macht die Einheit des berechneten Ergebnisses sichtbar:

```
// Näherung ohne Schaltjahre und genaues Geburtsdatum.  
int alterInJahren = 20;  
int alterInTagen = alterInJahren * 365;  
Console.WriteLine(alterInTagen);
```

Variablennamen werden üblicherweise in **camelCase** geschrieben: Der erste Buchstabe ist klein, weitere Wörter beginnen mit einem Großbuchstaben. Namen dürfen keine Leerzeichen enthalten. Reservierte Schlüsselwörter wie `int` sind als normale Variablennamen nicht erlaubt.

C# unterscheidet **Groß- und Kleinschreibung**. `alter` und `Alter` sind daher verschiedene Namen. Wird eine Variable unter einem anderen Namen gelesen als dem, unter dem sie deklariert wurde, kann der Compiler sie nicht zuordnen.

Mit `//` beginnt ein **Kommentar**, der bis zum Zeilenende reicht. Kommentare werden nicht ausgeführt. Sie helfen, die Absicht einer Berechnung oder eine bewusste Vereinfachung zu erklären. Im Beispiel macht der Kommentar deutlich, warum die Altersberechnung nur ungefähr ist.

7. Rechenoperatoren und Auswertungsreihenfolge

C# stellt die üblichen Rechenoperatoren zur Verfügung:

Operator	Bedeutung	Beispiel	Ergebnis
+	Addition	<code>10 + 3</code>	<code>13</code>
-	Subtraktion	<code>10 - 3</code>	<code>7</code>
*	Multiplikation	<code>10 * 3</code>	<code>30</code>
/	Division	<code>10 / 3</code>	<code>3</code>
%	Rest bei Ganzzahldivision	<code>10 % 3</code>	<code>1</code>

Multiplikation, Division und Restberechnung werden vor Addition und Subtraktion ausgewertet. Mit Klammern lässt sich eine andere Reihenfolge festlegen:

```
Console.WriteLine(2 + 3 * 4); // 14: Zuerst 3 * 4, dann + 2.  
Console.WriteLine((2 + 3) * 4); // 20: Zuerst die geklammerte Summe.
```

Ganzzahl- und Gleitkommadivision

Das Ergebnis einer Division hängt von den Typen der beteiligten Werte ab:

```
Console.WriteLine(5 / 2);    // 2: Division zweier int-Werte.  
Console.WriteLine(5.0 / 2); // 2.5: Ein Wert hat den Typ double.
```

Bei der **Ganzzahldivision** wird der Nachkommateil abgeschnitten. Es wird nicht auf die nächste ganze Zahl gerundet. Bei negativen Zahlen geschieht das Abschneiden in Richtung null: $-5 / 2$ ergibt -2 .

Eine häufige Fehlerquelle ist der Zieltyp der Zuweisung:

```
double ergebnis = 5 / 2;  
Console.WriteLine(ergebnis); // 2
```

Obwohl `ergebnis` den Typ `double` besitzt, wird rechts zuerst eine Ganzzahldivision durchgeführt. Erst deren Ergebnis `2` wird in einen `double`-Wert umgewandelt und gespeichert. Für ein Ergebnis mit Nachkommastellen muss bereits an der Berechnung ein passender Typ beteiligt sein, etwa bei `double ergebnis = 5.0 / 2;`

Restberechnung mit %

Der Operator `%` liefert den Rest einer Ganzzahldivision. Ein Beispiel ist die Verteilung von 23 Artikeln auf Kartons mit jeweils 5 Plätzen:

```
int artikel = 23;  
int proKarton = 5;  
  
int volleKartons = artikel / proKarton;  
int restlicheArtikel = artikel % proKarton;  
  
Console.WriteLine(volleKartons);    // 4  
Console.WriteLine(restlicheArtikel); // 3
```

Vier Kartons sind vollständig gefüllt, drei Artikel bleiben übrig. Die Berechnung liefert die Anzahl der **vollständig gefüllten Kartons**, nicht die Gesamtzahl der benötigten Kartons. Bei 25 Artikeln ergeben sich fünf volle Kartons und der Rest null.

Die Restberechnung eignet sich auch zum Prüfen der Teilbarkeit: Bleibt bei der Division durch zwei kein Rest, ist eine ganze Zahl gerade.

8. Eingaben lesen und in Zahlen umwandeln

Bisher stehen die verwendeten Werte fest im Programm. Mit `Console.ReadLine()` kann ein Programm stattdessen eine Eingabe entgegennehmen:

```
Console.WriteLine("Wie heißen Sie?");  
string name = Console.ReadLine();  
Console.WriteLine("Guten Tag.");  
Console.WriteLine(name);
```

`ReadLine()` wartet, bis eine Textzeile eingegeben und mit Enter bestätigt wurde. Anschließend wird der eingegebene Text in `name` gespeichert und das Programm fortgesetzt. Die Run-Konfiguration muss dafür interaktive Eingaben erlauben.

Ein möglicher Konsolenablauf lautet:

```
Wie heißen Sie?  
> Mia  
Guten Tag.  
Mia
```

Das Zeichen `>` kennzeichnet in diesem Skript eine Benutzereingabe. Es wird weder vom Beispielprogramm ausgegeben noch mit eingetippt. `WriteLine()` erzeugt eine Ausgabe, `ReadLine()` liefert eine Eingabe.

Die Beispiele setzen eine tatsächlich eingegebene Textzeile voraus. Bei aktiviertem Nullable-Checking kann der Editor darauf hinweisen, dass `ReadLine()` am Ende des Eingabestroms auch `null` liefern kann. Dieser Sonderfall und die robuste Eingabepfung werden später behandelt.

Text ist noch keine Zahl

Auch wenn `20` eingetippt wird, liefert `ReadLine()` zunächst den Text `"20"`. Für eine Zahlenberechnung muss dieser Text umgewandelt werden:

```
Console.WriteLine("Wie alt sind Sie in Jahren?");  
string alterAlsText = Console.ReadLine();  
int alterInJahren = Convert.ToInt32(alterAlsText);  
  
int alterNaechstesJahr = alterInJahren + 1;  
Console.WriteLine(alterNaechstesJahr);
```

Bei der Eingabe `20` wird `21` ausgegeben. Die Namen `alterAlsText` und `alterInJahren` verdeutlichen, an welcher Stelle Text und an welcher Stelle eine ganze Zahl verwendet wird.

`Convert.ToInt32(...)` wandelt einen geeigneten Text in einen `int`-Wert um. Das ist eine ausdrücklich ausgeführte Umwandlung; eine Zuweisung von `string` an `int` erfolgt nicht automatisch.

Für die ersten Übungen werden **gültige Eingaben** vorausgesetzt. Texte wie `zwanzig`, eine leere Eingabe oder Zahlen außerhalb des `int`-Bereichs können beim Umwandeln einen Laufzeitfehler auslösen. Verfahren zum Prüfen solcher Eingaben werden später eingeführt.

9. Fehler erkennen und beheben

Fehler gehören zum Programmieren. Entscheidend ist, zu erkennen, an welcher Stelle sie entstehen und welche Information bei der Suche hilft. Drei grundlegende Fehlerarten lassen sich unterscheiden:

Fehlerart	Verhalten	Beispiel
Compilerfehler	Der Code kann nicht erfolgreich übersetzt werden.	Ein fehlendes <code>;</code> , ein falsch geschriebener Variablenname oder ein unpassender Typ.
Laufzeitfehler	Das Programm startet, scheitert aber während der Ausführung.	<code>Convert.ToInt32(...)</code> erhält den Text <code>zwanzig</code> .
Logischer Fehler	Das Programm läuft, liefert aber ein falsches Ergebnis.	Ein Gesamtpreis wird nach einer Mengenänderung nicht neu berechnet.

Beginnen Sie bei einem Compilerfehler mit der ersten Fehlermeldung und der angegebenen Stelle. Prüfen Sie Namen, Datentypen, Anführungszeichen, Klammern und Semikolon. Ein einzelner Fehler kann weitere Meldungen verursachen, die nach seiner Korrektur verschwinden.

Bei unerwarteten Ergebnissen hilft es, das Programm Schritt für Schritt durchzugehen: Welche Werte besitzen die Variablen vor der Rechnung? Welche Anweisung verändert sie? Zwischenwerte lassen sich vorübergehend mit `Console.WriteLine(...)` ausgeben.

Arbeiten Sie in kleinen Schritten: **Ändern** → **speichern** → **starten** → **Ergebnis prüfen**. So lässt sich leichter feststellen, welche Änderung ein bestimmtes Verhalten verursacht hat.

10. Vergleiche und Wahrheitswerte

Programme müssen häufig entscheiden, ob eine Bedingung erfüllt ist. Dafür werden Werte verglichen. Das Ergebnis eines Vergleichs ist ein **Wahrheitswert** vom Typ `bool`:

```
int alter = 20;
bool istVolljaehrig = alter >= 18;
Console.WriteLine(istVolljaehrig); // Ausgabe: True
```

Der Ausdruck `alter >= 18` prüft, ob der aktuelle Wert mindestens 18 beträgt. Für `20` ergibt das `true`. Im Code werden die Schlüsselwörter `true` und `false` kleingeschrieben; `Console.WriteLine` gibt die Werte als `True` beziehungsweise `False` aus.

Operator	Bedeutung	Beispiel mit <code>alter = 20</code>	Ergebnis
<code>==</code>	Gleich	<code>alter == 18</code>	<code>false</code>
<code>!=</code>	Ungleich	<code>alter != 18</code>	<code>true</code>
<code><</code>	Kleiner	<code>alter < 18</code>	<code>false</code>
<code>></code>	Größer	<code>alter > 18</code>	<code>true</code>

Operator	Bedeutung	Beispiel mit <code>alter = 20</code>	Ergebnis
<code><=</code>	Kleiner oder gleich	<code>alter <= 20</code>	<code>true</code>
<code>>=</code>	Größer oder gleich	<code>alter >= 18</code>	<code>true</code>

Achtung: `=` weist einen Wert zu, `==` vergleicht zwei Werte. `alter = 18;` verändert die Variable. `alter == 18` prüft dagegen ihren aktuellen Wert.

11. Entscheidungen mit `if` und `else`

Die bisherigen Programme führen ihre Anweisungen der Reihe nach aus. Mit `if` („wenn“) und `else` („sonst“) lässt sich auswählen, welcher Code ausgeführt wird:

```
int alter = 20;

if (alter >= 18)
{
    Console.WriteLine("Volljährig");
}
else
{
    Console.WriteLine("Minderjährig");
}

Console.WriteLine("Prüfung beendet.");
```

Die Bestandteile sind:

- **`if (...)`:** Die runden Klammern enthalten eine Bedingung, deren Ergebnis `bool` ist.
- **Der `if`-Block:** Die Anweisungen zwischen den zugehörigen geschweiften Klammern werden ausgeführt, wenn die Bedingung `true` ergibt.
- **Der `else`-Block:** Seine Anweisungen werden ausgeführt, wenn die Bedingung `false` ergibt.
- **Die gemeinsame Fortsetzung:** Nach dem gewählten Block geht es hinter der Entscheidung weiter.

Bei einem `if/else` wird pro Ausführung der Entscheidung **genau einer der beiden Blöcke** ausgeführt. Die Bedingung prüft die Werte, die an dieser Stelle vorliegen. Sie wartet nicht darauf, dass sich ein Wert später verändert.

Zwei Programmläufe nachvollziehen

Für `alter = 20` wird `20 >= 18` geprüft. Die Bedingung ergibt `true`; der `if`-Block wird ausgeführt und der `else`-Block übersprungen:

```
Volljährig
Prüfung beendet.
```

Wird das Programm mit `alter = 16` erneut gestartet, ergibt `16 >= 18` den Wert `false`. Nun wird der `if`-Block übersprungen und der `else`-Block ausgeführt:

```
Minderjährig
Prüfung beendet.
```

Die letzte Ausgabe gehört zu keinem der beiden Blöcke und erscheint daher in beiden Fällen. Für den Grenzwert `18` ergibt die Bedingung ebenfalls `true`. Gerade solche Grenzwerte sind wichtige Testfälle.

Klammern, Einrückung und Semikolon

Geschweifte Klammern `{ ... }` fassen Anweisungen zu einem **Block** zusammen. Die Einrückung macht die Zugehörigkeit sichtbar; festgelegt wird sie durch die Klammern.

```
int alter = 16;

if (alter >= 18)
{
    Console.WriteLine("Volljährig");
    Console.WriteLine("Erwachsenentarif");
}

Console.WriteLine("Diese Ausgabe folgt immer.");
```

`else` ist optional. In diesem Beispiel ist die Bedingung falsch, sodass beide Anweisungen im Block übersprungen werden. Nur die letzte Ausgabe erscheint.

Direkt nach `if (...)` oder `else` steht kein Semikolon. Die Ausgabeanweisungen innerhalb des Blocks werden dagegen jeweils mit `;` beendet. Ein Semikolon direkt hinter `if (...)` würde einen leeren Anweisungskörper bilden und kann dazu führen, dass der nachfolgende Block unabhängig von der Bedingung ausgeführt wird.

In den Beispielen werden auch bei einer einzelnen Anweisung geschweifte Klammern verwendet. Eine innerhalb eines Blocks deklarierte Variable kann nur dort und in enthaltenen inneren Blöcken verwendet werden. Dieser Bereich heißt **Gültigkeitsbereich** oder **Scope**. Wird eine Variable auch nach der Entscheidung benötigt, sollte sie vorher angelegt werden.

12. Logische Operatoren: Bedingungen verknüpfen

Manche Entscheidungen hängen von mehreren Voraussetzungen ab. Logische Operatoren verknüpfen Wahrheitswerte oder kehren sie um:

Operator	Bedeutung	Beispiel
<code>&&</code>	Und: Beide Teilbedingungen müssen wahr sein.	<code>(alter >= 18) && hatTicket</code>

Operator	Bedeutung	Beispiel
<code> </code>	Oder: Mindestens eine Teilbedingung muss wahr sein.	<code>hatTicket stehtAufGaesteliste</code>
<code>!</code>	Nicht: Kehrt den Wahrheitswert um.	<code>!hatTicket</code>

Dabei sind `hatTicket` und `stehtAufGaesteliste` Variablen vom Typ `bool`. Das logische Oder schließt den Fall ein, dass **beide** Bedingungen wahr sind.

Erster Wert	Zweiter Wert	Ergebnis mit <code>&&</code>	Ergebnis mit <code> </code>
<code>false</code>	<code>false</code>	<code>false</code>	<code>false</code>
<code>false</code>	<code>true</code>	<code>false</code>	<code>true</code>
<code>true</code>	<code>false</code>	<code>false</code>	<code>true</code>
<code>true</code>	<code>true</code>	<code>true</code>	<code>true</code>

Der Operator `!` steht vor dem Wert oder dem geklammerten Ausdruck, den er verneint: `!true` ergibt `false`, `!false` ergibt `true`.

Beispiel: Zutritt prüfen

Die Regel lautet: Zutritt ist ab 18 Jahren erlaubt, wenn ein Ticket vorliegt oder die Person auf der Gästeliste steht.

```
int alter = 20;
bool hatTicket = false;
bool stehtAufGaesteliste = true;

if ((alter >= 18) && (hatTicket || stehtAufGaesteliste))
{
    Console.WriteLine("Zutritt erlaubt");
}
else
{
    Console.WriteLine("Kein Zutritt");
}
```

Die Teilbedingungen lassen sich gedanklich einzeln auswerten:

1. `alter >= 18` ergibt `true`.
2. `hatTicket || stehtAufGaesteliste` ergibt `false || true`, also `true`.
3. `true && true` ergibt `true`.

Das Programm gibt daher `Zutritt erlaubt` aus. Bei einem Alter von `17` wäre die Gesamtbedingung falsch, auch wenn sowohl ein Ticket als auch ein Gästelisteneintrag vorhanden wären.

Die äußeren runden Klammern gehören zur Schreibweise von `if`. Die inneren Klammern gruppieren die Teilbedingungen. Ohne zusätzliche Klammern bindet `!` stärker als `&&`, und `&&` stärker als `||`. Ausdrückliche Gruppierung hilft, die beabsichtigte Regel zu lesen.

Jeder Vergleich muss vollständig geschrieben werden. Für einen Altersbereich ist beispielsweise `(alter >= 18) && (alter <= 65)` gültig. Die verkürzte Schreibweise `alter >= 18 && <= 65` ist kein gültiger C#-Ausdruck.

`&&` und `||` besitzen außerdem eine **Kurzschlussauswertung**: Die rechte Seite wird nur ausgewertet, wenn sie für das Ergebnis noch benötigt wird. Ist bei `&&` die linke Seite bereits falsch, steht das Ergebnis fest. Ist bei `||` die linke Seite bereits wahr, steht es ebenfalls fest.

13. Ein vollständiges Beispiel: Alter in Tagen

Die bisher eingeführten Bestandteile lassen sich nun zu einem kleinen Programm verbinden:

```
Console.WriteLine("Wie alt sind Sie in Jahren?");
string alterAlsText = Console.ReadLine();
int alterInJahren = Convert.ToInt32(alterAlsText);

int alterInTagen = alterInJahren * 365;
Console.WriteLine("Ungefähres Alter in Tagen:");
Console.WriteLine(alterInTagen);

if (alterInJahren < 18)
{
    Console.WriteLine("Minderjährig");
}
else
{
    Console.WriteLine("Volljährig");
}
```

Die **Eingabe** liefert das Alter zunächst als Text. Die **Verarbeitung** wandelt diesen Text in eine ganze Zahl um, berechnet die ungefähre Tageszahl und prüft die Altersgrenze. Die **Ausgabe** zeigt das Ergebnis und den passenden Text.

Zum Prüfen eignen sich mehrere Werte:

Eingabe in Jahren	Ungefähres Alter in Tagen	Entscheidung
17	6205	Minderjährig
18	6570	Volljährig
20	7300	Volljährig

Die Aufgaben 1–8 im zugehörigen Aufgabenblatt üben diese Grundlagen: Programme starten, passende Datentypen auswählen, Werte kopieren und verändern, rechnen, Eingaben verarbeiten und

Entscheidungen treffen. Sagen Sie vor dem Ausführen möglichst die Ausgabe voraus und erklären Sie anschließend, welche Anweisungen zu ihr geführt haben.

14. Zusammenfassung des Kernteils

Die wichtigsten Erkenntnisse lassen sich wie folgt zusammenfassen:

- Ein **Programm** beschreibt konkrete Anweisungen. Ein **Algorithmus** ist eine genaue Schrittfolge zur Lösung einer Aufgabe.
- Eine **Variable** besitzt einen Datentyp, einen Namen und einen aktuellen Wert. Ein **Literal** ist ein direkt im Code geschriebener Wert.
- Eine **Zuweisung** wertet zuerst die rechte Seite aus und speichert das Ergebnis anschließend links. Gespeicherte Berechnungsergebnisse ändern sich erst durch eine neue Zuweisung.
- Die Typen der beteiligten Werte beeinflussen eine Berechnung: `5 / 2` ergibt `2`, während `5.0 / 2` den Wert `2.5` ergibt.
- `Console.WriteLine()` gibt Werte aus. `Console.ReadLine()` liest Text ein, der für Zahlenberechnungen ausdrücklich umgewandelt werden muss.
- Vergleiche liefern **bool-Werte**. `=` weist zu; `==` vergleicht.
- **if** und **else** wählen anhand einer Bedingung einen Codeblock aus. `&&`, `||` und `!` verknüpfen oder verneinen Bedingungen.
- Kleine Änderungen und gezielte Testfälle helfen, Compilerfehler, Laufzeitfehler und logische Fehler zu finden.

15. Optionale Vertiefung

Die folgenden Themen ergänzen den Kernteil. Sie dienen als Material für zusätzliche Übungszeit oder zum späteren Nacharbeiten und entsprechen den optionalen Folien sowie den Aufgaben 9–12.

Text und Werte gemeinsam ausgeben

Bisher wurden Texte und Variablen häufig in getrennten Zeilen ausgegeben. Mit **String-Interpolation** können Werte direkt in einen Text eingesetzt werden:

```
string name = "Mia";  
int alter = 20;  
  
Console.WriteLine($"{name} ist {alter} Jahre alt.");
```

Die Ausgabe lautet:

```
Mia ist 20 Jahre alt.
```

Das Dollarzeichen `$` steht vor dem öffnenden Anführungszeichen. Die geschweiften Klammern markieren die einzusetzenden Werte; sie gehören hier zur Textgestaltung und begrenzen keinen Codeblock. Ohne `$` würden die Namen und die Klammern wörtlich ausgegeben.

Dezimalzahlen einlesen und formatieren

Für eine Eingabe mit Nachkommastellen kann `Convert.ToDouble(...)` verwendet werden. Ein einfacher Währungsrechner zeigt die Umwandlung und eine formatierte Ausgabe:

```
Console.WriteLine("Betrag in Euro?");
string betragAlsText = Console.ReadLine();
double euro = Convert.ToDouble(betragAlsText);

double wechsellkurs = 1.08;
double dollar = euro * wechsellkurs;

Console.WriteLine($"{euro:0.00} Euro entsprechen {dollar:0.00} US-
Dollar.");
```

Der Wechselkurs `1.08` ist ein fester, frei gewählter **Übungskurs**. Für die Eingabe `10` ergibt die Berechnung `10.8`; die Formatangabe `:0.00` stellt das Ergebnis mit zwei Nachkommastellen dar. Eine mögliche Ausgabe ist `10.00 Euro entsprechen 10.80 US-Dollar.`

Formatierung verändert die Darstellung, nicht den gespeicherten Wert. Außerdem muss zwischen Quellcode und Eingabe unterschieden werden: Im Quellcode steht immer ein Dezimalpunkt. Bei `Convert.ToDouble(...)` hängt das akzeptierte Eingabeformat von den Spracheinstellungen der Laufzeitumgebung ab. Je nach Umgebung wird beispielsweise `10.50` oder `10,50` erwartet; auch die Ausgabe kann Punkt oder Komma verwenden.

Für die Übung werden passende Eingaben vorausgesetzt. Die gezielte Steuerung des Zahlenformats und die Prüfung ungültiger Eingaben werden später vertieft. Für echte Geldberechnungen ist der bereits eingeordnete Typ `decimal` relevant.

Typableitung mit `var`

Bei lokalen Variablen kann der Compiler den Typ aus dem Startwert ableiten:

```
var alter = 20;    // Der Compiler bestimmt den Typ int.
var laenge = 1.75; // Der Compiler bestimmt den Typ double.
var name = "Mia";  // Der Compiler bestimmt den Typ string.
```

`var` ist eine alternative Schreibweise für eine Deklaration mit Startwert. Es ist kein zusätzlicher Datentyp. Der abgeleitete Typ bleibt fest: `alter` kann anschließend weiterhin nur passende Ganzzahlwerte speichern. Eine Deklaration wie `var alter;` ohne Startwert funktioniert nicht, weil der Compiler daraus keinen Typ bestimmen kann.

Zu Beginn sind ausdrücklich angegebene Typen hilfreich, weil sich die Verbindung zwischen Variable und Wert direkt am Code ablesen lässt.

Kurzformen von Zuweisungen

Für häufige Veränderungen gibt es kürzere Schreibweisen:

```
int punkte = 10;

punkte += 5; // Hier gleichbedeutend mit: punkte = punkte + 5;
punkte -= 2; // Hier gleichbedeutend mit: punkte = punkte - 2;
punkte++;    // Erhöht den Wert um 1.

Console.WriteLine(punkte); // Ausgabe: 14
```

Nach den Anweisungen besitzt `punkte` nacheinander die Werte `15`, `13` und `14`. Entsprechende Kurzformen existieren auch für Multiplikation und Division, etwa `*=` und `/=`. In dieser Einführung wird `++` nur als eigene Anweisung verwendet.

Mehrere Fälle mit `else if`

Wenn mehr als zwei Fälle unterschieden werden sollen, kann eine Entscheidung um `else if` erweitert werden:

```
int alter = 16;

if (alter < 6)
{
    Console.WriteLine("Eintritt frei");
}
else if (alter < 18)
{
    Console.WriteLine("Ticket: 5 Euro");
}
else
{
    Console.WriteLine("Erwachsenentarif");
}
```

Die Bedingungen werden der Reihe nach geprüft. Die **erste zutreffende Bedingung** bestimmt den ausgeführten Block; spätere Bedingungen werden danach nicht mehr geprüft. Das abschließende `else` behandelt die übrigen Fälle.

Für `16` ist `alter < 6` falsch und `alter < 18` wahr. Daher wird `Ticket: 5 Euro` ausgegeben. Beim Prüfen der zweiten Bedingung ist bereits bekannt, dass das Alter mindestens sechs beträgt. Eine zusätzliche Prüfung dieser unteren Grenze ist deshalb hier nicht nötig.

Die Reihenfolge ist entscheidend: Würde zuerst `alter < 18` geprüft, würde diese Bedingung auch für Kinder unter sechs Jahren zutreffen. Mit den Werten `5`, `6`, `17` und `18` lassen sich die Übergänge zwischen den Fällen prüfen. Aufgabe 10 erweitert dieses Muster um einen weiteren Tarif ab 65 Jahren.

Weitere Anwendungen: Schaltjahr und Login-Simulation

Die optionalen Aufgaben 11 und 12 kombinieren bekannte Bestandteile in etwas umfangreicheren Entscheidungen.

Beim **Schaltjahr-Rechner** wird die vorgegebene Regel zunächst in Teilbedingungen zerlegt: durch vier teilbar, durch hundert teilbar und durch vierhundert teilbar. Teilbarkeit wird jeweils mit `%` und einem Vergleich des Restes geprüft. Erst danach werden die Teilbedingungen passend mit `&&`, `||` und Klammern verbunden. Die Jahre `2000`, `2024`, `1900` und `2023` prüfen unterschiedliche Fälle der Regel.

Bei der **Login-Simulation** werden eingegebene Texte mit festgelegten Texten verglichen. `==` kann hier prüfen, ob beispielsweise der eingegebene Name gleich `"admin"` ist. Diese Textvergleiche berücksichtigen die Groß- und Kleinschreibung. Eine weitere `if/else`-Entscheidung im passenden Namensfall prüft anschließend das Passwort. Eine solche Entscheidung innerhalb eines anderen Blocks wird als **verschachtelt** bezeichnet.

Die Login-Simulation dient dem Üben von Textvergleichen und Programmabläufen. Ein echtes Login erfordert weitere Verfahren, insbesondere für die Speicherung und Prüfung von Passwörtern.